TraceWeave User Guide

> Version: v1.1.0 | A beginner-friendly user guide

If you have ever hit these problems:

- "Which assets use this material?"
- "Why can't I delete this texture?"
- "Which scenes or assets reference this Prefab?"
- "I know there is a reference, but I can't tell which object, component, or field it lands on."

`TraceWeave` solves exactly that. It is not a generic search box — it is a **reverse-reference tracing tool**.

What it does in one sentence

Give it a target asset, and it finds:

- what this asset directly depends on (references)
- which assets in the project reference it back (referenced-by)
- and, when resolvable, exactly where the hit lands:
- hierarchy path
- component
- field

Think of it as: "pick a target, see who references it, and locate the exact hit site."

Best use cases

- verify impact before deleting an asset
- understand the blast radius before replacing a material
- trace how a texture / material / Prefab propagates through the project
- know whether a hit lives in a scene, a Prefab, or another asset file

How to open it

Two ways:

1. Menu: `Window/TLNEXUS/TraceWeave`
2. Select one or more assets in the Project window, right-click → `Assets/TLNEXUS/Open in TraceWeave` (batch-adds multiple targets)

Quick start (new workflow)

No manual Scan button — scanning starts automatically when a target is added.

1. Select the asset(s) you want to investigate in the Project window
2. Right-click → `Open in TraceWeave` — the assets land in the left **target list**

- Or open the window and simply **drag assets** onto the target list

1. Scanning starts automatically; progress is shown at the top
2. Click a node in the graph, read the hit details on the right

The target list supports **multiple targets**: add several assets and click to switch. Each target's results are cached, so switching back is instant.

Reading the window

Four areas:

- **Top toolbar**: status text + progress bar + percent + `Rebuild Index` + `Settings`
- **Left**: target list (add / switch / refresh / clear targets)
- **Middle**: search field + category summary + reference graph (main result view)
- **Right**: hit details (where the match actually lands)

Core concepts

1. Target

The assets in the left target list are what you are investigating. The meaning is not "export this", but:

"Find who references this."

Good candidates: materials, textures, Prefabs, ScriptableObjects, scene assets.

2. References (outgoing)

The left column of the graph shows what the target directly depends on. A Prefab target, for example, may depend on materials, textures, and animations.

3. Referenced By (incoming)

The right column of the graph shows which assets reference the target. This is usually the part you actually want — "who is using it".

4. Hit details

After clicking a node, the right pane shows:

- the asset name, type, and path
- whether the hit is a direct dependency, a field reference, or a scene hit
- when resolvable: hierarchy path, component name, and field path

Reading the reference graph

- **Center node**: the target asset (accent-colored border, most visible)
- **Left column**: "References (N)" — outgoing dependencies
- **Right column**: "Referenced By (N)" — incoming references
- Curved lines with arrowheads show reference direction

Graph interactions:

- **Scroll wheel**: zoom (0.55x – 2.75x)
- **Middle-mouse drag**: pan
- **Click a node**: select it; the right pane updates
- **Double-click a node**: open that asset
- **Click empty space**: clear the selection

Filtering results

1. Search field

The input at the top right of the middle pane filters results by name or path (tooltip: "Filter results").

2. Category summary tiles

A row of category tiles: Material / Prefab / Scene / Scriptable / Animation / Shader / Texture / Model / Audio / Other.

- Each tile shows the match count for that category
- **Click a tile to show/hide that category** (disabled tiles turn semi-transparent)
- Note: this filters results **after** the scan — it is not a scan scope

Reading the right detail pane

Top to bottom after selecting a node:

- **Selection**: label
- **Title / Type**: category | concrete type name
- **Path**: full asset path
- **Hit explanation**:
- Direct dependency node: "Direct dependency of the target asset."
- Scene hit: "Scene dependency detected."
- Field references: "{N} detected field reference(s)"
- **Actions**: `Ping` / `Open` / `Copy Path` / `Select` / `Copy Hierarchy`
- `Ping`: locate in the Project window (if an AssetClip window is open, pings inside it first)
- `Open`: open the asset
- `Copy Path`: copy the asset path
- `Select`: select the asset
- `Copy Hierarchy`: copy the matched hierarchy paths (batched)
- **Detected Fields**: hit rows / hit tree
- Shown as a tree when hierarchy paths exist (enable "Guide Lines" to visualize nesting)
- Shown as plain rows when only field paths exist
- At most 24 hits are stored; more are summarized as "{N} more field reference(s) not expanded"
- **Two toggles**:
- `Highlight Matches`: emphasize matched hierarchy nodes
- `Guide Lines`: draw guide lines in the hierarchy tree

Reading hierarchy paths

A resolved hit looks like:

```
Canvas/GamePlayPanel/Btn_Start/Icon
  └ Image component
      └ m_Sprite field
```

Translation: "The Image component on Btn_Start in this scene references the target via its Sprite field."

How scene and Prefab hits are resolved

- **Prefab hits**: the Prefab is loaded, then every transform/component is scanned for serialized field references to reconstruct hierarchy paths
- **Scene hits**: if the scene is not open, it is opened additively for inspection (the previous state is restored afterwards); if the target itself is a Prefab, all Prefab instances in the scene are located
- Objects that cannot be serialized-inspected (e.g. missing references) still keep a valid dependency result; their hierarchy path may show as unresolved

Project reference badges

After enabling "Project Reference Badges" in Settings, each asset in the Project window shows its reference count in the corner (capped at "999+").

Badge data comes from the reference index. Its status is shown below the toolbar:

- **Disabled**: badges off; the index will not run automatically
- **Outdated**: project changed; rebuild the index manually
- **Rebuilding**: shows scanned X/Y assets
- **Up to date**: N assets (Prefabs X | Scenes Y | Other Z)

Important: index rebuild is explicit. When the project changes, the index is only marked outdated — it never rebuilds silently in the background. Click `Rebuild Index` in the toolbar to rebuild (a dialog confirms completion). This is intentional: transparent and controllable, no hidden background work.

Settings panel

Opened via the gear button on the toolbar:

- **Language**: Chinese / English (persisted)
- **Project Reference Badges**: toggle
- **About**: version, author, homepage, documentation link, tool description

Common workflows

1. Check impact before deleting

1. Add the asset to the target list (context menu or drag)
2. Wait for the scan, read the "Referenced By" column
3. If there are active references, do not delete it yet

2. Understand propagation before replacing a material

1. Target the material
2. See who references it; click nodes and read the object/field hits on the right

3. Find which scenes a Prefab lands in

1. Target the Prefab
2. Look at scene nodes in the "Referenced By" column and read their hierarchy paths

Common misconceptions

1. **It is not a full-project text search.** It traces references around a chosen target.
2. **"References" is not "Referenced By".** Left column = what the target uses; right column = who uses the target.

3. **Do not read only the graph.** The graph says *who* matched; the right pane says *where* it matched (hierarchy, component, field).
4. **Do not click through results blindly.** Narrow with category tiles first, then the search field, then inspect details.
5. **Do not wait for the badge index to update itself.** Rebuilds are explicit — when it shows "outdated", click `Rebuild Index`.

Technical notes (for advanced users)

- **Unified reference index**: one build pass produces two views — `path → referencing asset list` (scan) and `guid → reference count` (badges) — persisted to a single snapshot so the two views can never disagree
- **Snapshot file**: `Library/TLNEXUS/TraceWeave/reference-index.json`
- **Incremental builds**: dependency results are cached by file write time; only changed assets are re-resolved, so everyday refreshes take ~1–3 seconds
- **Time-sliced execution**: scanning and indexing run in per-frame time budgets (~0.02s), keeping the editor responsive
- **Dirtiness tracking**: project changes, undo/redo, scene saves, and object change events mark the index outdated; indexing is suspended in Play mode
- **Graph render budget**: at most 480 connections rendered (180 while panning); larger result sets are strided automatically
- **Settings file**: `Library/TLNEXUS/TraceWeave/settings.json`

One-line summary

"Add targets (context menu or drag) → scan starts automatically → read the graph (left = references, right = referenced-by) → read the right pane for exact hit locations."

Homepage: www.tlnexus.com Author: T·L Email: 455471553@qq.com